Enhancing Code LLM Training with Programmer Attention

Yifan Zhang
Vanderbilt University
Nashville, USA

Chen Huang
Sichuan University
Chengdu, China

Zachary Karas
Vanderbilt University
Nashville, USA

Dung Thuy Nguyen
Vanderbilt University
Nashville, USA

Kevin Leach
Vanderbilt University
Nashville, USA

Yu Huang
Vanderbilt University
Nashville, USA

Abstract

Human attention provides valuable yet underexploited signals for code LLM training, offering a perspective beyond purely machine-driven attention. Despite the complexity and cost of collecting eye-tracking data, there has also been limited progress in systematically *using* these signals for code llm training. To address both issues, we propose a cohesive pipeline spanning *augmentation* and *reward-based fine-tuning*. Specifically, we introduce (1) an *eye-tracking path augmentation* method to expand programmer attention datasets, (2) a *pattern abstraction* step that refines raw fixations into learnable attention motifs, and (3) a *reward-guided* strategy for integrating these insights directly into a CodeT5 supervised fine-tuning process. Our experiments yield +7.16 in CodeBLEU on the CodeXGlue benchmark for code summarization, underscoring how uniting human and machine attention can boost code intelligence. We hope this work encourages broader exploration of human-centric methods in next-generation AI4SE.

CCS Concepts

• **Software and its engineering** → **Automatic programming**; • **Information systems** → **Document representation**.

Keywords

Large Language Models, Eye Tracking, Code Comprehension

ACM Reference Format:
Yifan Zhang, Chen Huang, Zachary Karas, Dung Thuy Nguyen, Kevin Leach, and Yu Huang. 2025. Enhancing Code LLM Training with Programmer Attention. In *33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25), June 23–28, 2025, Trondheim, Norway.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3696630.3728510

1 Introduction

Programmers often exhibit selective attention when interpreting and modifying source code. Understanding these real-world attention patterns can deepen our grasp of code comprehension, improving tasks like summarization or completion [11, 21]. However, collecting such data (for example, via eye tracking or manual annotations) is expensive and time-consuming, limiting its large-scale use. Even when available, novel ways of integrating human attention signals into LLM training remain limited [7, 15], so most AI-driven code tools rely on static artifacts alone. This gap underscores the need for data-augmentation strategies and specialized pipelines that systematically embed genuine programmer fixations into LLM-based code intelligence.

While LLMs like CodeT5 improve when fine-tuned on large code corpora, integrating human signals like eye-tracking attention patterns can enhance them further [15, 33]. However, typical data augmentation often overlooks code's cognitive demands or specific structures [7, 9]. Consequently, even advanced LLMs may underutilize valuable programmer attention information (like scanpaths or reading orders), especially when such data is sparse. Effectively embedding these developer signals necessitates specialized training pipelines. To address this, we propose HumanLLM, a two-part approach primarily for code summarization (also tested on completion and translation). First, HumanLLM uses real eye-tracking data reflecting programmer reading to augment code. Second, it embeds these signals via reward-based fine-tuning of CodeT5 [30], aligning the model with human fixations. This systematic use of human-centric patterns aims to bridge cognitive insights and automated code intelligence across tasks.

Although our attention patterns originate from code summarization studies and may not fully generalize to completion or translation, our experiments show that injecting these human-centric signals can notably improve model performance. For instance, we observed a up to +7.16 in CodeBLEU [23], +2.67 in Syntax, and +14.97 in Dataflow on the CodeXGlue dataset for code summarization, suggesting the potential of attention-driven methods to enhance software engineering workflows. This reinforces the vision that systematically integrating real programmer attention into code LLM training can unify cognitive insights with automated solutions across diverse AI4SE tasks.

2 Preliminaries

Our two-part code LLM fine-tuning pipeline leverages three key preliminaries to integrate developer attention:

Eye-Tracking for Programming. Eye tracking captures *fixations*, where developers pause on tokens, and *saccades*, the rapid jumps between them, revealing real-time cognitive strategies in code reading [20, 25]. These insights show how programmers parse syntax and identify relevant constructs, grounding our human-centered pipeline (Section 3).

This work is licensed under a Creative Commons Attribution 4.0 International License.
FSE Companion '25, Trondheim, Norway
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1276-0/2025/06
https://doi.org/10.1145/3696630.3728510

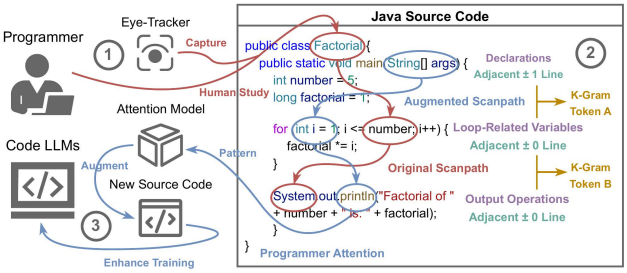

Figure 1: Overview of our pipeline (HumanLLM): ① collects eye-tracking data (red) to capture real programmer attention; ② augments these fixations with AST-based adjacency (blue) and k-gram patterns; and ③ uses these human signals to guide a reward-based CodeT5 fine-tuning.

Data Augmentation in AI for Code. Traditional augmentation methods (e.g., token substitutions, AST edits) often overlook developers' natural reading flows [3, 31]. By integrating actual eye-tracking data (Section 3.2), our approach captures these real patterns, helping models learn cognitively grounded cues that reflect authentic developer attention.

Code LLM Training. Modern code LLMs typically follow *Retrieval-Augmented Generation (RAG)* or *Supervised Fine-Tuning (SFT)* [7, 10]. We employ a *reward-guided SFT* procedure, periodically injecting eye-tracking signals into the training loop (Section 3.3), so that model outputs better match actual programmer fixations.

3 Approach

Figure 1 outlines our pipeline, which we refer to as HumanLLM, integrating eye-tracking data into a CodeT5 model in three stages. First, ① *Data Collection* gathers token-level fixation data from an open Java eye-tracking dataset [14] and supplements it with Java snippets from CodeXGlue[1]. Second, ② *Augmentation* enriches these fixations with AST-based semantic labels, adjacency expansions, k-gram patterns, and positional ordering (Sections 3.1–3.2). Finally, ③ *Reward-Based Fine-Tuning* periodically fuses these human-guided signals into CodeT5 training (Section 3.3), aligning the model with genuine programmer behavior.

3.1 Data Collection

We use the open eye-tracking dataset [14], which captures how 15 professional developers read and summarized 120 Java methods (30–50 lines each), logging token-level fixations and saccades. To enhance diversity, we incorporate Java samples from CodeXGlue. Each snippet provides *code tokens* (e.g., `if`, `UserInput`, `42`), mapped to *semantic tokens* (`Keyword`, `Identifier`, `Literal`) via AST annotations. These fixation traces and mappings serve as the foundation for augmentation and reward-based training.

[1] https://github.com/microsoft/CodeXGLUE

3.2 Human-Centric Augmentation of Code Tokens

Let $\mathbf{T} = \{t_1, \ldots, t_n\}$ be the tokens in a Java snippet, and let $\mathbf{F} \subset \mathbf{T}$ represent those actually fixated by developers. We construct an enriched set $\mathbf{F}^\star$ that encodes the following programmer attention information:

Semantic Labeling. We parse each method's AST and assign a label $L(t_i)$ (e.g., Keyword, Literal) to each token $t_i \in \mathbf{T}$. This step ensures that the model recognizes syntactic roles rather than relying on raw text.

Adjacency-Based Expansion. For each $t_i \in \mathbf{F}$, we include any tokens ±3 lines away that share $L(t_i)$. Formally,

$$\mathbf{F}^\star \leftarrow \mathbf{F} \cup \left\{ t_j \,\middle|\, |\text{line}(t_j) - \text{line}(t_i)| \le 3 \ \wedge \ L(t_j) = L(t_i) \right\}.$$

This threshold of three lines stems from our analysis of the open eye-tracking dataset [14], showing that over 95% of attention shifts occur within this distance, indicating that developers often check nearby lines with similarly categorized tokens.

K-Gram Patterns. Prior work on developer behavior [14, 33] shows that bigrams ($k = 2$) and trigrams ($k = 3$) reflect how programmers cluster code tokens. We detect these patterns in $\mathbf{F}^\star$, retain the top 20 for reward modeling, and assign each p_ℓ a numeric label. If a snippet matches both, we prioritize the trigram, breaking ties by frequency. Capturing these reading clusters models how developers group tokens during comprehension.

Positional Ordering. Beyond adjacency expansions and k-gram patterns, we track each token's reading index $\pi(t_i)$ to align augmented tokens with actual scanpaths. This captures how developers encounter tokens in sequence, extending beyond simple locality. We record positions for up to 100 tokens per snippet, ensuring consistent fixation mapping. Each token $t_i \in \mathbf{F}^\star$ is labeled with $\big(p_\ell(t_i), \pi(t_i)\big)$, encoding its semantic pattern and reading order for a more human-guided code representation.

3.3 Reward-Based Fine-Tuning with Programmer Attention

We integrate these augmented signals into a CodeT5 pipeline that periodically references the human-informed dataset $\mathbf{F}^\star$. After every m mini-batches ($m = 20$) of standard cross-entropy (CE) training, we sample a mini-batch from $\mathbf{F}^\star$, generate predictions $\hat{y}$, and compare them to the ground-truth labels $y^\star$. The resulting reward $\mathcal{R}(\hat{y}, y^\star)$ measures how well the model's predicted sequences align with genuine developer fixations (e.g., matching k-grams or positional order). We then form a combined loss

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \alpha \mathcal{R},$$

where α regulates the strength of human guidance. By repeatedly backpropagating $\nabla \mathcal{L}_{\text{total}}$, CodeT5 learns to minimize typical prediction error while conforming to real programmer attention patterns. Over multiple epochs, these reward passes steer the model to internalize adjacency relationships, k-gram fixations, and realistic scanpath ordering, thereby producing more human-aligned outputs (e.g., code summaries).

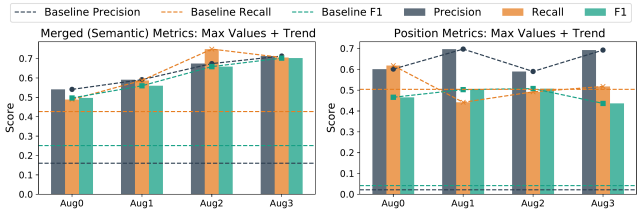

Figure 2: Impact of adjacency expansions (0 to 3 lines) on semantic (left) and positional (right) labels. Wider windows generally improve Precision, Recall, and F1, exceeding a baseline Transformer (dashed lines) in five of six metrics.

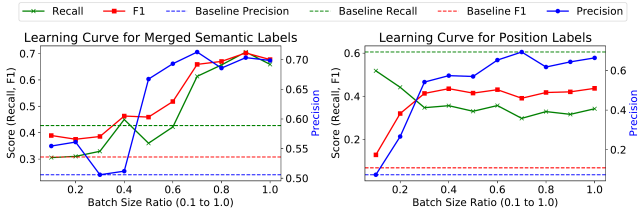

Figure 3: Learning curves for semantic (left) and positional (right) labels over one epoch, shown via the batch-size ratio. Dashed lines represent the baseline Transformer. Larger ratios correlate with better test-set performance.

4 Experimental Design

We conduct two sets of experiments: (1) a basic Transformer analysis on an 80–20 split of our augmented eye-tracking dataset to assess whether the enriched signals are genuinely learnable, and (2) a CodeT5-based evaluation (using `codet5-base`[2]) on three tasks in CodeXGlue for Java (summarization, completion, and Java-to-C# translation).

In the first setting, we train for a single epoch, tracking progress (0–100%) via the *batch-size ratio* to ensure the Transformer absorbs semantic (k-gram) and positional signals. In the second, we fine-tune CodeT5 with a reward-based loop (every 20 batches), adding an $\mathcal{R}$-driven term to cross-entropy that aligns predictions with real developer fixations. We fix seed 42 for data sampling and initialization, train on two NVIDIA A6000 GPUs (48 GB each), and use AdamW (lr $= 5 \times 10^{-5}$) with gradient checkpointing.

To balance diversity and runtime constraints, we sample 10% of each CodeXGlue dataset. We report BLEU, ROUGE-L, and METEOR for textual fidelity in code summarization, and CodeBLEU for deeper code-level accuracy (incorporating syntax and dataflow checks). This combination of metrics provides a comprehensive measure of both linguistic coherence and program correctness.

5 Experimental Results

We structure our analysis in three parts: (1) validating augmented signals across different adjacency windows and batch-size schedules, (2) showing how these signals improve CodeT5's summarization (both NLP and code metrics), and (3) testing their transfer to completion and translation.

5.1 Learning with Human-Centric Augmentations

Adjacency Window Effects. Figure 2 reports Precision, Recall, and F1 on a held-out test set for semantic and positional labels when expanding adjacency from 0 to 3 lines. In five of the six metrics, performance surpasses a baseline Transformer (dashed lines). Only Recall for positional labels lags slightly at the widest window, indicating that too much context can dilute the reading-order signal.

Batch-Size Ratio (Training Progress). Figure 3 then examines how performance evolves during a single epoch, where the *batch-size ratio* on the horizontal axis reflects progress from 0% to 100% of that epoch. Larger ratios generally yield higher F1, Precision, and most Recall values, suggesting that the model benefits from encountering more human-centric samples earlier and more frequently.

Together, Figures 2 and 3 confirm that adjacency expansions and sufficient training exposure help the Transformer internalize real programmer fixations.

5.2 Enhanced Code Summarization

We next assess whether these cues boost code summarization. Table 1 shows notable gains in BLEU, ROUGE-L, and METEOR, while Table 2 reveals parallel improvements in CodeBLEU, Syntax, and Dataflow (up by nearly 15 points). This suggests that real developer fixations help the model track variables and data dependencies, mirroring human comprehension.

Table 1: Summarization (NLP Metrics). HumanLLM integrates programmer attention into CodeT5, resulting in more fluent, context-aware summaries.

Pipeline	BLEU	ROUGE-L	METEOR
Baseline	5.36	10.80	7.22
HumanLLM	10.32	14.91	12.88
Change	+4.96	+4.11	+5.66

Table 2: Summarization (Code Metrics). By leveraging actual fixations, HumanLLM demonstrates improved structural understanding and variable handling.

Pipeline	CodeBLEU	Syntax	Dataflow
Baseline	8.51	3.06	18.01
HumanLLM	15.67	5.73	32.98
Change	+7.16	+2.67	+14.97

Figures 4 and 5 show that these improvements extend across different function types. HumanLLM outperforms the baseline in syntax and data flow for most categories, indicating the augmented signals help CodeT5 adapt to varied coding styles.

[2] https://github.com/salesforce/CodeT5

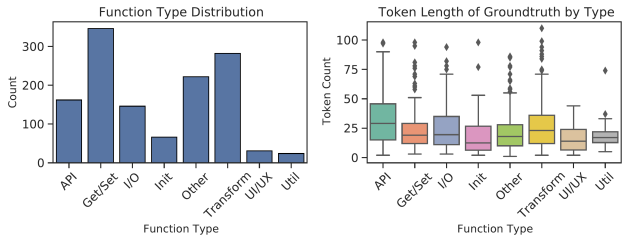

Figure 4: Overview of function types (left) and token-length distribution (right) for the ground-truth snippets in our dataset. The histogram on the left shows how frequently each function type appears, while the box plot on the right illustrates variability in token counts across different types.

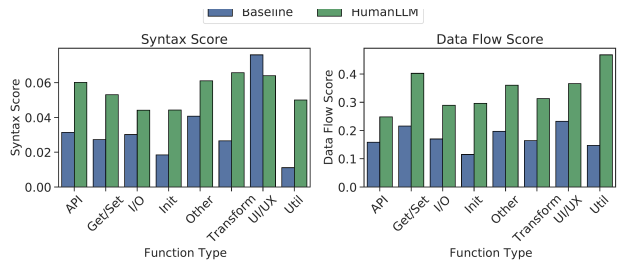

Figure 5: Syntax (left) and Data Flow (right) scores by function type for Baseline vs. HumanLLM. HumanLLM generally outperforms the Baseline by better capturing control structures and variable interactions, aligning more closely with real developer fixations.

5.3 Task-Specificity for Completion and Translation

Table 3 shows results on code completion and Java-to-C# translation, where syntax alignment sees minor gains but CodeBLEU and dataflow do not improve substantially. This highlights the specialization of our method for summarization: these fixation patterns do not transfer easily to tasks that rely on different developer attention cues. Future extensions may require distinct eye-tracking protocols or reward definitions to optimize performance in completion, translation, or other AI4SE scenarios.

Table 3: Completion and Translation (Code Metrics). Summarization-centric attention signals offer limited gains for other tasks.

Task	Pipeline	CodeBLEU	Syntax	Dataflow
Completion	Baseline	5.42	10.15	3.40
	HumanLLM	4.77	10.46	1.45
	Change	-0.65	+0.31	-1.95
Translation	Baseline	15.18	19.38	14.50
	HumanLLM	14.94	18.96	14.38
	Change	-0.24	-0.42	-0.12

6 Future Work

Our observations using eye tracking data from professional developers show promising improvements in code summarization metrics. In future work, we will develop novel augmentation techniques to better leverage these signals and extend benefits to tasks such as code review, fault localization, and security auditing. By incorporating richer eye tracking data and flexible reward strategies, we aim to build AI driven coding assistants that capture developer focus and deepen code understanding.

7 Threat to Validity

Our study is based on an eye tracking dataset from professional Java developers, which may limit the generalizability of our findings to other languages and paradigms. The use of 10% of the CodeXGlue data may introduce sampling biases and, although we fix the random seed (42), some variation remains. Our reward scheme is designed for code summarization and may require adjustments for tasks such as bug detection, refactoring, or translation. Finally, the resource intensive nature of gathering eye tracking data calls for more efficient methods to leverage these signals.

8 Related Work

We briefly survey how code intelligence integrates human-centered signals, review neural approaches to code comprehension, and note why many LLMs overlook real programmer focus. Our method closes these gaps by infusing eye-tracking data into Code LLM training.

Human-Centered AI for Software Engineering. Human input aids SE tasks (e.g., developer annotations [9, 12, 13], or crowdsourcing [19]). Eye tracking reveals detailed reading strategies [4, 14, 18, 27], yet most work applies these signals post hoc [5] or in narrow domains [33]. We directly integrate eye-tracking insights into LLM fine-tuning to capture genuine developer scanpaths.

Neural Code Analysis and Comprehension. Deep learning powers code summarization, clone detection, and bug identification [6, 17, 24, 26, 32], often using tokens, ASTs, or graphs [2, 22, 28, 34]. These methods typically ignore real developer focus and rely on static artifacts. Our approach augments models with human attention, adding a more developer-centric lens to code comprehension.

LLMs for Code. Models such as CodeBERT [8], Codex [7], and CodeT5 [29, 30] excel via large-scale pretraining and selective fine-tuning but often miss actual programmer fixations. Although some studies consider integrating human attention [1, 16, 18], efforts remain limited. Our reward-guided framework systematically aligns the model with genuine developer

9 Conclusion

We present a developer-attention–driven approach for fine-tuning Code LLMs, aligning model outputs with real programmer fixations using eye-tracking data. By augmenting these signals with semantic labels and positional cues and applying reward-based fine-tuning to CodeT5, we achieve notable gains in textual and code-specific metrics. Our results underscore the potential of integrating human insight with AI, fostering deeper collaboration in AI4SE.

References

[1] Muhammad Abulaish, Nesar Ahmad Wasi, and Shachi Sharma. 2024. The role of lifelong machine learning in bridging the gap between human and machine learning: A scientometric analysis. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 14, 2 (2024), e1526.

[2] Manish Acharya, Yifan Zhang, Yu Huang, and Kevin Leach. 2025. Optimizing Code Runtime Performance through Context-Aware Retrieval-Augmented Generation. *arXiv preprint arXiv:2501.16692* (2025).

[3] Miltiadis Allamanis, Earl T Barr, Premkumar Devanbu, and Charles Sutton. 2018. A survey of machine learning for big code and naturalness. *ACM Computing Surveys (CSUR)* 51, 4 (2018), 1–37.

[4] Aakash Bansal, Bonita Sharif, and Collin McMillan. 2023. Towards modeling human attention from eye movements for neural source code summarization. *Proceedings of the ACM on Human-Computer Interaction* 7, ETRA (2023), 1–19.

[5] Aakash Bansal, Chia-Yi Su, Zachary Karas, Yifan Zhang, Yu Huang, Toby Jia-Jun Li, and Collin McMillan. 2023. Modeling programmer attention as scanpath prediction. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1732–1736.

[6] Aakash Bansal, Chia-Yi Su, and Collin McMillan. 2023. Revisiting File Context for Source Code Summarization. *arXiv preprint arXiv:2309.02326* (2023).

[7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[8] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).

[9] Muhammad Hamza, Dominik Siemon, Muhammad Azeem Akbar, and Tahsinur Rahman. 2024. Human-ai collaboration in software engineering: Lessons learned from a hands-on workshop. In *Proceedings of the 7th ACM/IEEE International Workshop on Software-intensive Business*. 7–14.

[10] Pengfei He, Shaowei Wang, Shaiful Chowdhury, and Tse-Hsun Chen. 2024. Exploring Demonstration Retrievers in RAG for Coding Tasks: Yeas and Nays! *arXiv preprint arXiv:2410.09662* (2024).

[11] Muntasir Hoq, Jessica Vandenberg, Bradford Mott, James Lester, Narges Norouzi, and Bita Akram. 2024. Towards Attention-Based Automatic Misconception Identification in Introductory Programming Courses. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 2*. 1680–1681.

[12] Chen Huang, Yang Deng, Wenqiang Lei, Jiancheng Lv, Tat-Seng Chua, and Jimmy Xiangji Huang. 2025. How to Enable Effective Cooperation Between Humans and NLP Models: A Survey of Principles, Formalizations, and Beyond. *arXiv preprint arXiv:2501.05714* (2025).

[13] Chen Huang, Peixin Qin, Wenqiang Lei, and Jiancheng Lv. 2023. Reduce Human Labor On Evaluating Conversational Information Retrieval System: A Human-Machine Collaboration Approach. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 10876–10891.

[14] Zachary Karas, Aakash Bansal, Yifan Zhang, Toby Li, Collin McMillan, and Yu Huang. 2024. A Tale of Two Comprehensions? Analyzing Student Programmer Attention during Code Summarization. *ACM Transactions on Software Engineering and Methodology* (2024).

[15] Bonan Kou, Shengmai Chen, Zhijie Wang, Lei Ma, and Tianyi Zhang. 2024. Do large language models pay similar attention like human programmers when generating code? *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 2261–2284.

[16] Qiuxia Lai, Salman Khan, Yongwei Nie, Hanqiu Sun, Jianbing Shen, and Ling Shao. 2020. Understanding more about human and machine attention in deep neural networks. *IEEE Transactions on Multimedia* 23 (2020), 2086–2099.

[17] Jiliang Li, Yifan Zhang, Yu Huang, and Kevin Leach. 2024. Malmixer: Few-shot malware classification with retrieval-augmented semi-supervised learning. *arXiv preprint arXiv:2409.13213* (2024).

[18] Jiliang Li, Yifan Zhang, Zachary Karas, Collin McMillan, Kevin Leach, and Yu Huang. 2024. Do Machines and Humans Focus on Similar Code? Exploring Explainability of Large Language Models in Code Summarization. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension*. 47–51.

[19] Yao Lu, Song Bian, Lequn Chen, Yongjun He, Yulong Hui, Matthew Lentz, Beibin Li, Fei Liu, Jialin Li, Qi Liu, et al. 2024. Computing in the Era of Large Generative Models: From Cloud-Native to AI-Native. *arXiv preprint arXiv:2401.12230* (2024).

[20] Roberto Minelli, Andrea Mocci, and Michele Lanza. 2015. I know what you did last summer-an investigation of how developers spend their time. In *2015 IEEE 23rd international conference on program comprehension*. IEEE, 25–35.

[21] Erik Nijkamp, Hiroaki Hayashi, Caiming Xiong, Silvio Savarese, and Yingbo Zhou. 2023. Codegen2: Lessons for training llms on programming and natural languages. *arXiv preprint arXiv:2305.02309* (2023).

[22] Shankara Pailoor, Yuepeng Wang, and Işıl Dillig. 2024. Semantic code refactoring for abstract data types. *Proceedings of the ACM on Programming Languages* 8, POPL (2024), 816–847.

[23] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297* (2020).

[24] Cedric Richter, Jan Haltermann, Marie-Christine Jakobs, Felix Pauck, Stefan Schott, and Heike Wehrheim. 2022. Are Neural Bug Detectors Comparable to Software Developers on Variable Misuse Bugs?. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–12.

[25] Dario D Salvucci and Joseph H Goldberg. 2000. Identifying fixations and saccades in eye-tracking protocols. In *Proceedings of the 2000 symposium on Eye tracking research & applications*. 71–78.

[26] Ensheng Shi, Yanlin Wang, Lun Du, Junjie Chen, Shi Han, Hongyu Zhang, Dongmei Zhang, and Hongbin Sun. 2022. On the evaluation of neural code summarization. In *Proceedings of the 44th international conference on software engineering*. 1597–1608.

[27] Ningzhi Tang, Meng Chen, Zheng Ning, Aakash Bansal, Yu Huang, Collin McMillan, and Toby Jia-Jun Li. 2024. Developer behaviors in validating and repairing llm-generated code using ide and eye tracking. In *2024 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, 40–46.

[28] Xin Wang, Yasheng Wang, Fei Mi, Pingyi Zhou, Yao Wan, Xiao Liu, Li Li, Hao Wu, Jin Liu, and Xin Jiang. 2021. Syncobert: Syntax-guided multi-modal contrastive pre-training for code representation. *arXiv preprint arXiv:2108.04556* (2021).

[29] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi DQ Bui, Junnan Li, and Steven CH Hoi. 2023. Codet5+: Open code large language models for code understanding and generation. *arXiv preprint arXiv:2305.07922* (2023).

[30] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv preprint arXiv:2109.00859* (2021).

[31] Shiwen Yu, Ting Wang, and Ji Wang. 2022. Data augmentation by program transformation. *Journal of Systems and Software* 190 (2022), 111304.

[32] Yifan Zhang, Chen Huang, Kevin Cao, Yueke Zhang, Scott Thomas Andersen, Huajie Shao, Kevin Leach, and Yu Huang. 2022. Pre-Training Representations of Binary Code Using Contrastive Learning. *arXiv preprint arXiv:2210.05102* (2022).

[33] Yifan Zhang, Jiliang Li, Zachary Karas, Aakash Bansal, Toby Jia-Jun Li, Collin McMillan, Kevin Leach, and Yu Huang. 2024. Eyetrans: Merging human and machine attention for neural code summarization. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 115–136.

[34] Yifan Zhang, Junwen Yang, Haoyu Dong, Qingchen Wang, Huajie Shao, Kevin Leach, and Yu Huang. 2022. Astro: An ast-assisted approach for generalizable neural clone detection. *arXiv preprint arXiv:2208.08067* (2022).